

GUJARAT TECHNOLOGICAL UNIVERSITY

Chandkheda, Ahmedabad

Sardar Vallabhbhai Patel Institute of Technology, Vasad

A Project On

PBL Activity 3: Complex Problem Solving (CPS)

Smart Study Planner: An SDG-Oriented Personal Academic Planning Web Application

Under the Subject

Web Application Development (BE05000281)

BE 3rd YEAR, SEM 5 (Computer Engineering branch)

Submitted by:

Husain Bardanwala (240410107008)

Nikunj Dhobi (240410107033)

Prof. Nidhi Shah (Faculty Guide)

Prof. Jayna Shah (Head of the Department)

SARDAR VALLABHBHAI PATEL INSTITUTE OF TECHNOLOGY, VASAD

SUB NAME: Web Application Development

SUBJECT CODE: BE05000281

TABLE OF CONTENTS

Sr no.	Topic	Page no.
1	Title Page	3
2	Introduction	3
3	Project Definition & Problem Statement / Objectives	4
4	Proposed System / System Design	5
5	Screenshots of Results	9
6	Conclusion and Future Scope	11
7	References	11

1. Title Page

SMART STUDY PLANNER: AN SDG-ORIENTED PERSONAL ACADEMIC PLANNING WEB APPLICATION

A full-stack (React + Node.js + MongoDB) web application that helps students plan tasks, track study hours with a Pomodoro timer, and visualize progress to reduce missed deadlines.

Item	Details
Project title	Smart Study Planner: An SDG-Oriented Personal Academic Planning Web Application
Activity	PBL Activity 3 – Complex Problem Solving (CPS)
SDGs addressed	SDG 4 (Quality Education)
Subject	Web Application Development (BE05000281)
Technology stack	MongoDB, Express.js, React.js, Node.js (MERN)
Submitted by	Husain Bardanwala (240410107008), Nikunj Dhobi (240410107033)
Class	BE 3rd Year, Semester 5, Computer Engineering
Faculty guide	Prof. Nidhi Shah
Institute	Sardar Vallabhbhai Patel Institute of Technology, Vasad

2. Introduction

A web application is software that runs on a web server and is used through a web browser. Unlike a static website that only shows fixed pages, a web application accepts input from users, stores data in a database and shows each user information that matches their role. Modern web applications are usually split into a frontend, a backend, and a database.

The United Nations Sustainable Development Goals (SDGs) are 17 global goals for 2030. SDG 4 – Quality Education aims to ensure inclusive and equitable quality education and promote lifelong learning opportunities for all. Students who lack structured study habits often miss deadlines, cannot estimate time per subject, and rarely review progress.

Before this project, task information was scattered across WhatsApp messages, personal notebooks, and calendar apps that do not understand the academic workload. There was no single place to view subjects, tasks, deadlines, study hours, and progress together.

In this project, a web application named Smart Study Planner is designed and developed using the MERN stack. A student registers, logs in, creates subjects and tasks with due dates and priorities,

marks tasks complete, and uses a configurable Pomodoro timer to focus. The dashboard shows total, completed, pending tasks, subject count, and an overall progress bar.

Need of the project: centralize all academic tasks in one place, never miss a deadline again, visualize progress, and build consistent study habits with the Pomodoro timer.

Web technologies used in this project:

- HTML5 and CSS3 – semantic structure, Flexbox/Grid layouts and media queries for responsive design.
- JavaScript (ES6+) – arrow functions, destructuring, Promises and async/await for API calls.
- React.js – component-based SPA with hooks, routing and conditional rendering.
- Node.js and Express.js – REST API server with middleware for logging, authentication and error handling.
- MongoDB with Mongoose – NoSQL document database with schemas and validation.
- HTTP, JSON and REST – communication between browser and server, tested with Postman and curl.

3. Project Definition & Problem Statement / Objectives

3.1 Project Definition

Item	Details
PBL activity	Activity 3 – Complex Problem Solving (CPS)
Domain	Education / productivity (SDG-oriented web application)
Project title	Smart Study Planner – Academic Task & Time Manager
Users (roles)	Student
Frontend	React 19, Vite, React Router, CSS3
Backend	Node.js, Express.js, JSON Web Token (JWT), bcryptjs
Database	MongoDB with Mongoose ODM
Tools	VS Code, MongoDB Compass, Postman, npm, Git

3.2 Problem Statement

Students today juggle assignments, exams, projects, and revision across multiple platforms. They do not have a single place to see what is due, what is done, which subjects need more time, and how much progress they have made. Missing due dates leads to last-minute panic and lower scores. This project delivers a single-page dashboard that centralizes subjects and tasks, highlights overdue items, and encourages focused study with a Pomodoro timer.

3.3 Objectives

- 1. To design and develop a full-stack web application using React, Node.js, Express.js and MongoDB.

- 2. To provide secure registration and login with hashed passwords and JWT-based authentication.
- 3. To let students create, edit, complete and delete tasks with priorities and due dates.
- 4. To build REST APIs that perform all CRUD operations on tasks and subjects.
- 5. To show live dashboard stats and a progress bar based on completion.
- 6. To implement a configurable Pomodoro timer for focused study.
- 7. To make the interface responsive and usable on mobile phones and desktops.

3.4 Scope of the Project

The project covers user management, subject creation, task CRUD, filtering by subject/priority, deadline sorting, overdue highlighting, a progress bar, and a Pomodoro timer. Future enhancements include study-hour analytics, shared group lists, notifications, and calendar sync.

Live URLs: Frontend <https://smart-study-planner-pi-gules.vercel.app> | API <https://smartstudyplanner-api.onrender.com>

4. Proposed System / System Design

4.1 System Architecture

Smart Study Planner follows a three-tier client-server architecture. The React application runs in the browser and sends HTTP requests in JSON format. The Express server verifies the JWT token, runs business logic, and reads/writes data in MongoDB through Mongoose. The database stores users, tasks, and subjects.

Figure 1: System architecture of Smart Study Planner

4.2 Technologies Used

Layer	Technology	Purpose in project
Frontend	React 19 + Vite	Component-based SPA, fast dev server
Routing	React Router 6	Pages without full reload; protected routes
HTTP client	fetch API	API calls; Authorization header with JWT
Styling	CSS3 (Flexbox, Grid, media queries)	Responsive layout for mobile and desktop
Backend	Node.js + Express.js	REST API, routing and middleware
Security	bcryptjs, jsonwebtoken	Password hashing and token-based login
Database	MongoDB + Mongoose	Document storage, schema validation
Testing	Postman, curl, Playwright	API tests and UI smoke tests
Deployment	Render, Vercel, Atlas	Cloud hosting

4.3 Modules of the System

- Authentication module – register, log in, persist token in localStorage, log out.
- Dashboard module – stats cards, progress bar, quick actions.
- Task module – create, edit, toggle complete, delete, filter by subject/priority, sort by deadline, overdue badge.
- Subject module – add subjects and use them as task categories.
- Pomodoro timer module – configurable duration, start/pause/reset.

4.4 Database Design

Collection	Fields	Rules
users	name, email, password, createdAt, updatedAt	email unique, password bcrypt-hashed
tasks	name, subject, due, priority, completed, user, createdAt, updatedAt	name/ subject required, priority enum, user ref
subjects	name, user, createdAt, updatedAt	name required, user ref

4.5 REST API Design

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Register student
POST	/api/auth/login	Public	Login, receive JWT
GET	/api/auth/me	Logged in	Current user profile
GET	/api/tasks	Logged in	List my tasks
POST	/api/tasks	Logged in	Create task
PUT	/api/tasks/:id	Logged in	Update task (whitelisted fields)
DELETE	/api/tasks/:id	Logged in	Delete task
GET	/api/subjects	Logged in	List my subjects
POST	/api/subjects	Logged in	Add subject
DELETE	/api/subjects/:id	Logged in	Delete subject

4.6 Working of the System

1. The student registers/logs in. The server checks the password with bcrypt and returns a signed JWT, which React saves in localStorage.
2. The dashboard fetches tasks/subjects using the token in the Authorization header.
3. The student creates a task; React sends POST /api/tasks. Mongoose validates the data and saves it.
4. The student can mark a task complete, edit its details, delete it, or filter by subject/priority.
5. The Pomodoro timer runs in the browser and helps the student focus in configurable intervals.

4.7 Security Features

- Passwords hashed with bcrypt (10 salt rounds).
- JWT authentication: protected routes return 401 without a valid token.
- User ID is taken from the token, not the request body.
- Whitelisted update fields on tasks/subjects.
- Secrets in .env, excluded from Git.
- CORS origin-configurable; input trimming and length validation.

4.8 Implementation Steps

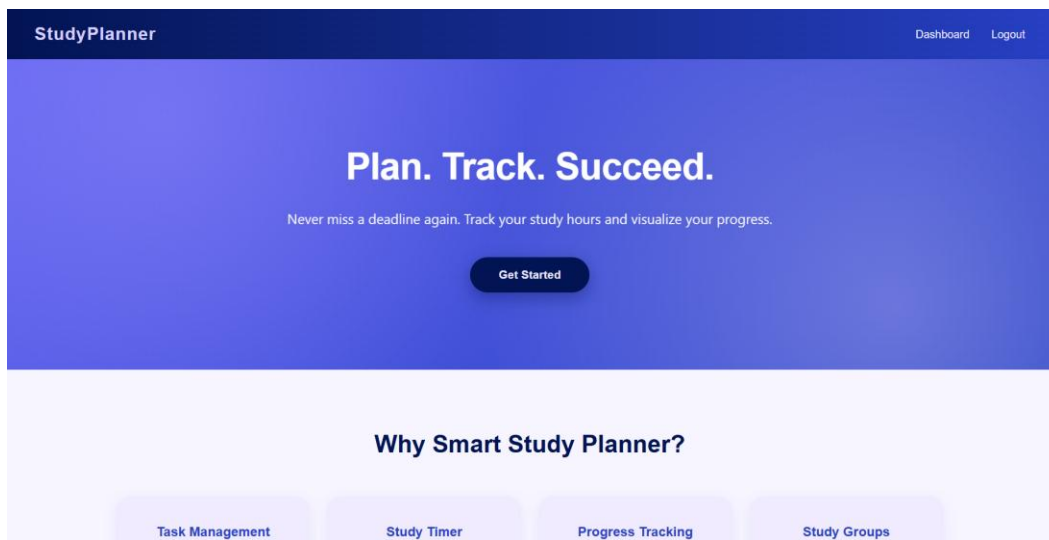
- Install Node.js, MongoDB Community Server, create React+Vite client.
- Build backend: npm init, express, mongoose, cors, dotenv, bcryptjs, jsonwebtoken.
- Create Mongoose models, middleware, routes in MVC-style structure.
- Build React pages: Home, Login, Register, Dashboard; components: Navbar, TaskCard, TaskForm, Timer, ProgressBar.
- Test APIs with Postman/curl; add seed + demo data.
- Deploy: Atlas for DB, Render for API, Vercel for frontend.

5. Screenshots of Results

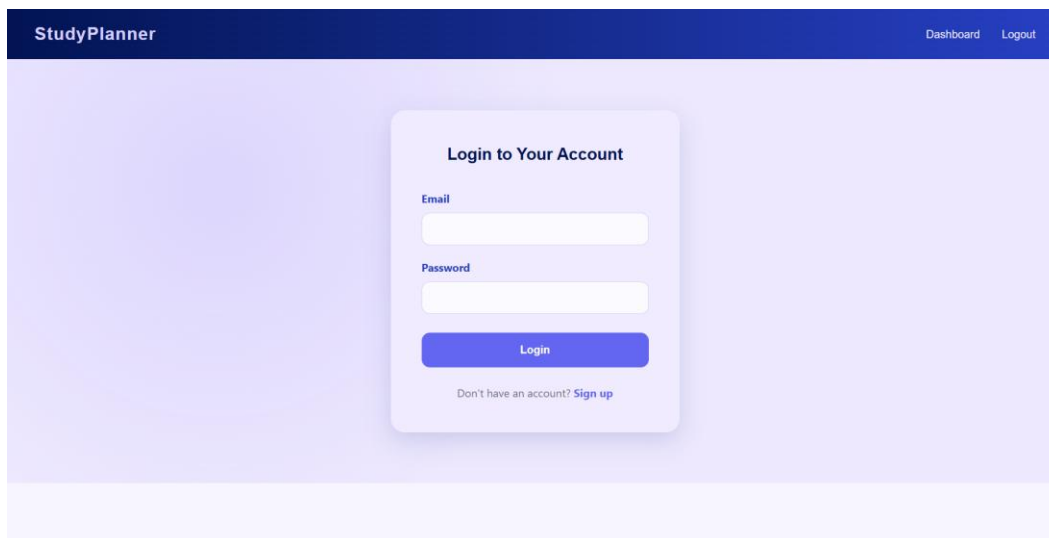
The application was run with demo data. All functional tests produced the expected result.

Test	Action	Expected result	Result
1	Open home page	Hero + features shown, no login required	Pass
2	Register new user	JWT returned, redirected to dashboard	Pass
3	Login as student	Token issued, dashboard opens	Pass
4	Add task	201 Created, appears in list	Pass
5	Mark complete	Status flips, progress bar updates	Pass
6	Edit task	Fields prefilled, PUT updates list	Pass
7	Filter by subject/priority	Only matching tasks shown	Pass
8	Start Pomodoro	Countdown runs, pause/reset works	Pass

Screenshot 1: Landing page with features



Screenshot 2: Login page



Screenshot 3: Dashboard with demo data and overdue task

Screenshot 4: Add task form

StudyPlanner

DashboardLogout

Welcome, Aarav Sharma!

TOTAL TASKS

7

COMPLETED

3

PENDING

4

SUBJECTS

5

Overall Progress

43%

43% of tasks completed

Quick Actions

+ Add Task

+ Add Subject

Start Timer

Add New Task

Biology Chapter 12 Notes

Biology

dd-mm-yyyy

Medium

Add

Recent Tasks

All Subjects

All Priorities

Sort by Deadline

English Essay - My School Day (English · low)

Due: 09/10/2026

DoneUndoEditDelete

Solve Chapter 5 - Integrals Ex 5.1 (Mathematics · high)

Due: 10/10/2026

DoneUndoEditDelete

Physics Lab File - Motion in 2D (Physics · high)

Due: 12/10/2026

DoneUndoEditDelete

Chemistry reactions notes Ch 8 (Chemistry · medium)

Due: 15/10/2026

PendingCompleteEditDelete

CS Practical - Write Python program for linked list (Computer Science · high)

Due: 18/10/2026

PendingCompleteEditDelete

Revise Organic Chemistry Ch 10 (Chemistry · medium)

Due: 20/10/2026

PendingCompleteEditDelete

Physics sample paper practice (Physics · medium)

Due: 25/10/2026

PendingCompleteEditDelete

Screenshot 5: Pomodoro timer with presets

Four blue cards displaying statistics:

- TOTAL TASKS**: 7
- COMPLETED**: 3
- PENDING**: 4
- SUBJECTS**: 5

A horizontal progress bar with a blue fill and a white border. The bar is 43% full. Below the bar, the text "43% of tasks completed" is displayed.

Task Completion Status	Percentage
Completed	43%
Pending	57%

[+ Add Task](#)
[+ Add Subject](#)
[Start Timer](#)

25:00

Minutes: 25 15 25 45 60 Set

Start

Pause

Reset

All Subjects ▾ All Priorities ▾ Sort by Deadline

Done

Undo

Edit

Delete

Done

Undo

Edit

Delete

Done

Undo

Edit

Delete

Pending

Complete

Edit

Delete

Pending

Complete

Edit

Delete

Pending

Complete

Edit

Delete

Pending

Complete

Edit

Delete

6. Conclusion and Future Scope

Smart Study Planner solves a real academic productivity problem. Students can centralize subjects and tasks, mark them done, see their progress, and focus better with a Pomodoro timer. It demonstrates the full web application stack: HTML/CSS, JavaScript, React, Node.js/Express, MongoDB, JWT security, and cloud deployment.

Future scope:

- Study-hour analytics and weekly charts.
- Notifications before deadlines.
- Group tasks for study teams.
- Calendar view and Google Calendar sync.
- Mobile app (PWA) support.

7. References

- United Nations, "Sustainable Development Goals – Goal 4", sdgs.un.org.
- React documentation, react.dev.
- Express.js documentation, expressjs.com.
- MongoDB and Mongoose documentation, mongodb.com/docs and mongoosejs.com.
- MDN Web Docs – HTML, CSS, JavaScript and HTTP, developer.mozilla.org.